SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1402 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks:100

Annexure A

EXPERIMENTS

Code of Conduct:

*I **KAMALESH M (192421438)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student

KAMALESH M

Annexure A

EXPERIMENTS

Q1.You are developing a compiler-based desk calculator that evaluates arithmetic expressions using syntax-directed definitions (SDD).

The system should:

(i) Accept an arithmetic expression with operators +, -, *, / and parentheses

(ii) Use syntax-directed evaluation to compute values based on operator precedence

(iii) Optimize evaluation by reducing unnecessary temporary computations

(iv) Display the final result of the expression

Demonstrate how SDD rules (synthesized attributes) are used for expression evaluation.

SDD

Grammar with semantic rules:

E → E + T { E.val = E1.val + T.val }
E → E - T { E.val = E1.val - T.val } E → T { E.val = T.val }

T → T * F { T.val = T1.val * F.val
} T → T / F { T.val = T1.val /
F.val } T → F { T.val =
F.val }

F → (E) { F.val = E.val
} F → digit { F.val = digit
}

Q2. In a compiler, postfix expressions are evaluated during code generation using stack-based bottom-up evaluation.
Write a C program that:

(i) Evaluates a postfix expression

(ii) Demonstrates S-attributed definition evaluation

(iii)Shows how intermediate results are computed

Q3.You are designing a parser that uses L-attributed definitions, where inherited attributes are passed from parent to child.
Write a C program that:
(i).Evaluates a simple expression like a + b * c
(ii) Uses function parameters to simulate inherited attributes
(iii). Ensures correct precedence handling
(iv). Demonstrates top-down evaluation logic
Explain how inherited attributes influence computation.

Q4.In a compiler, it is important to check whether two type expressions are equivalent.

Write a C program that:
(i).Accepts two type expressions (e.g., int, float, int*)

(ii).Compares them for **type equivalence**
(iii).Displays whether they are:
(a).Equivalent
(b).Not equivalent
Extend the logic to handle pointer types (int*, float*).

Q5.You are building a semantic analyzer that detects type errors in arithmetic expressions.
Write a C program that:
(i). Accepts two operands and their types
(ii). Checks whether the operation (e.g., +, *) is valid
(iii). Reports:
(a).Valid expression
(b).Type error (e.g., int + char*)
Demonstrate how compilers detect semantic errors during type checking

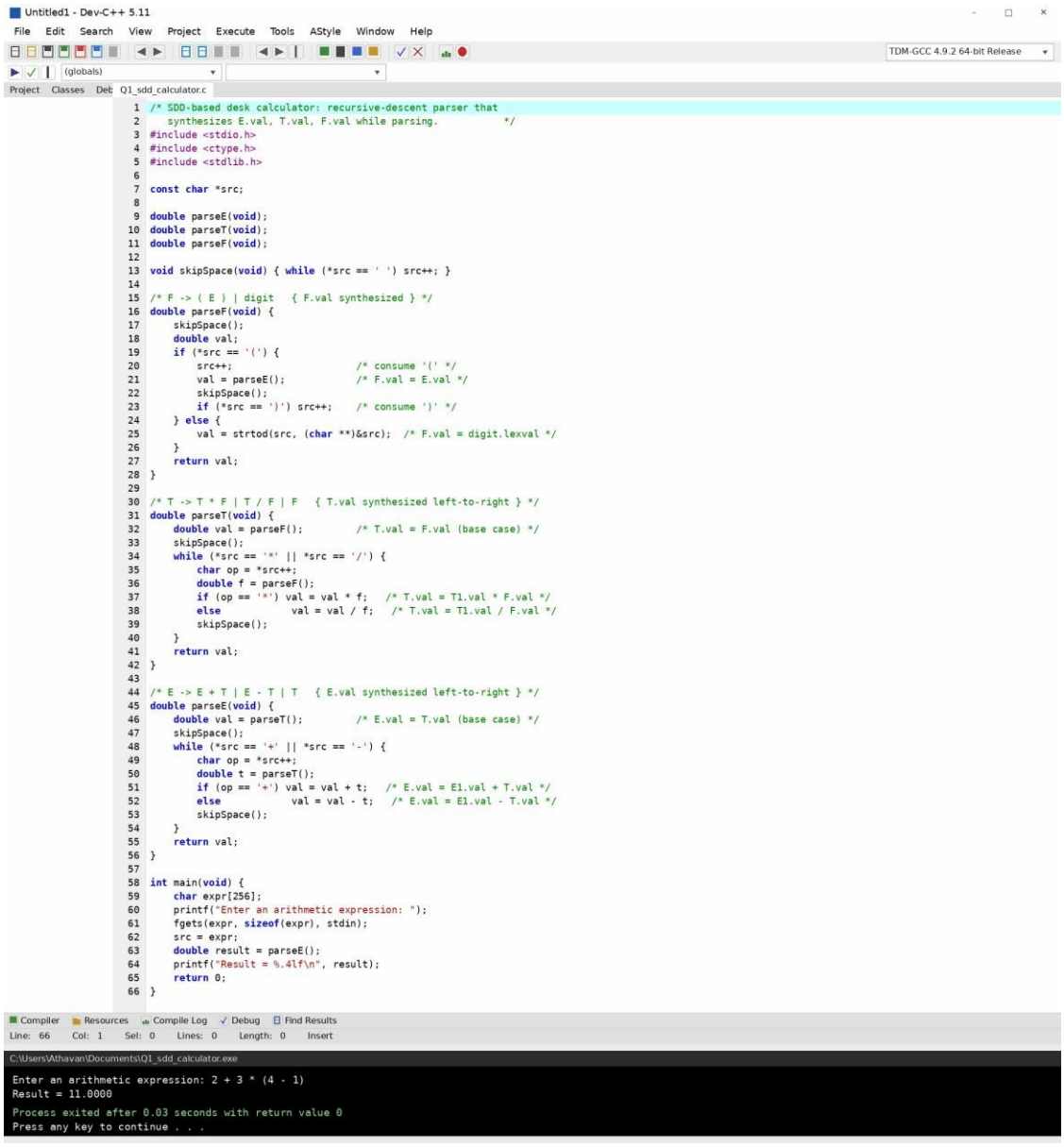
RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q1. Syntax-Directed Definition (SDD) for a Desk Calculator

- Accepts an arithmetic expression with +, -, *, / and parentheses.
- Uses synthesized attributes (E.val, T.val, F.val) to evaluate the expression honouring operator precedence.
- Evaluates each production's action immediately on reduction, avoiding unnecessary temporaries.

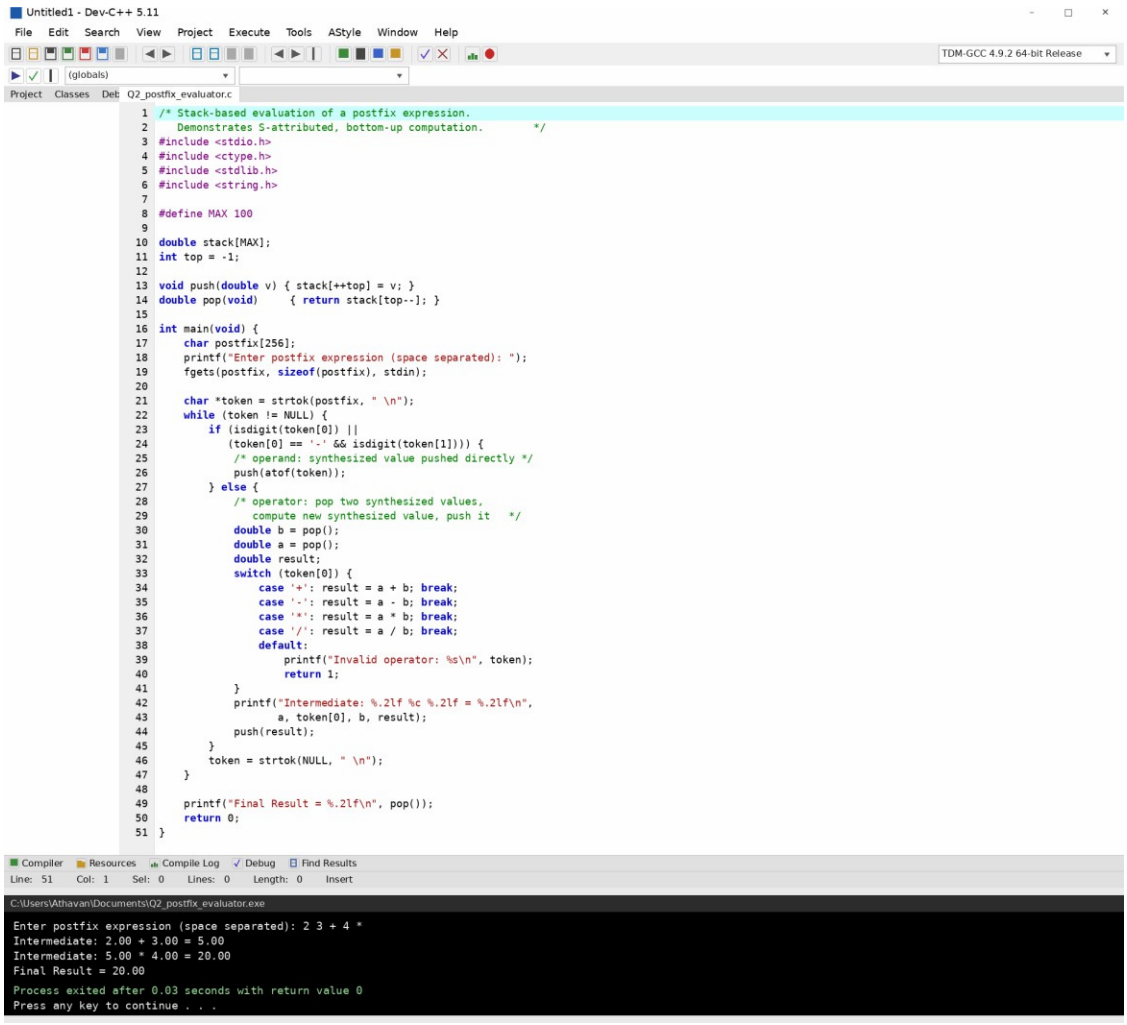
Program (Dev-C++) and Console Output:



Q2. Postfix Expression Evaluation (S-Attributed, Stack-Based Bottom-Up)

- Evaluates a postfix expression using a stack.
- Demonstrates S-attributed definition evaluation — every attribute is synthesized from children already reduced on the stack.
- Prints each intermediate result as it is computed.

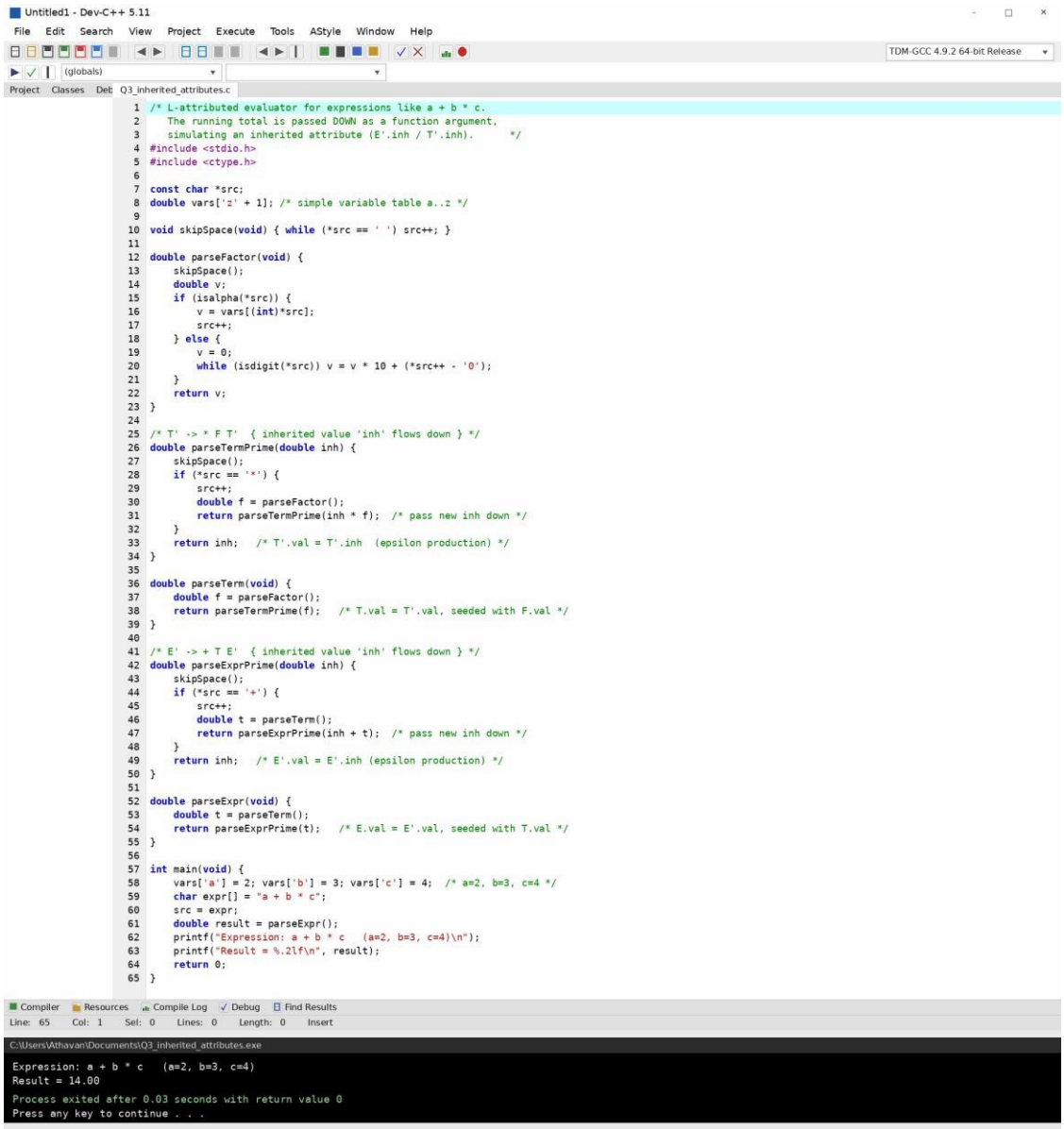
Program (Dev-C++) and Console Output:



Q3. L-Attributed Definition — Inherited Attributes (Top-Down Evaluation)

- Evaluates an expression like a + b * c.
- Uses function parameters to simulate inherited attributes (the running total is passed down the call chain).
- Ensures correct precedence handling and demonstrates top-down evaluation logic.

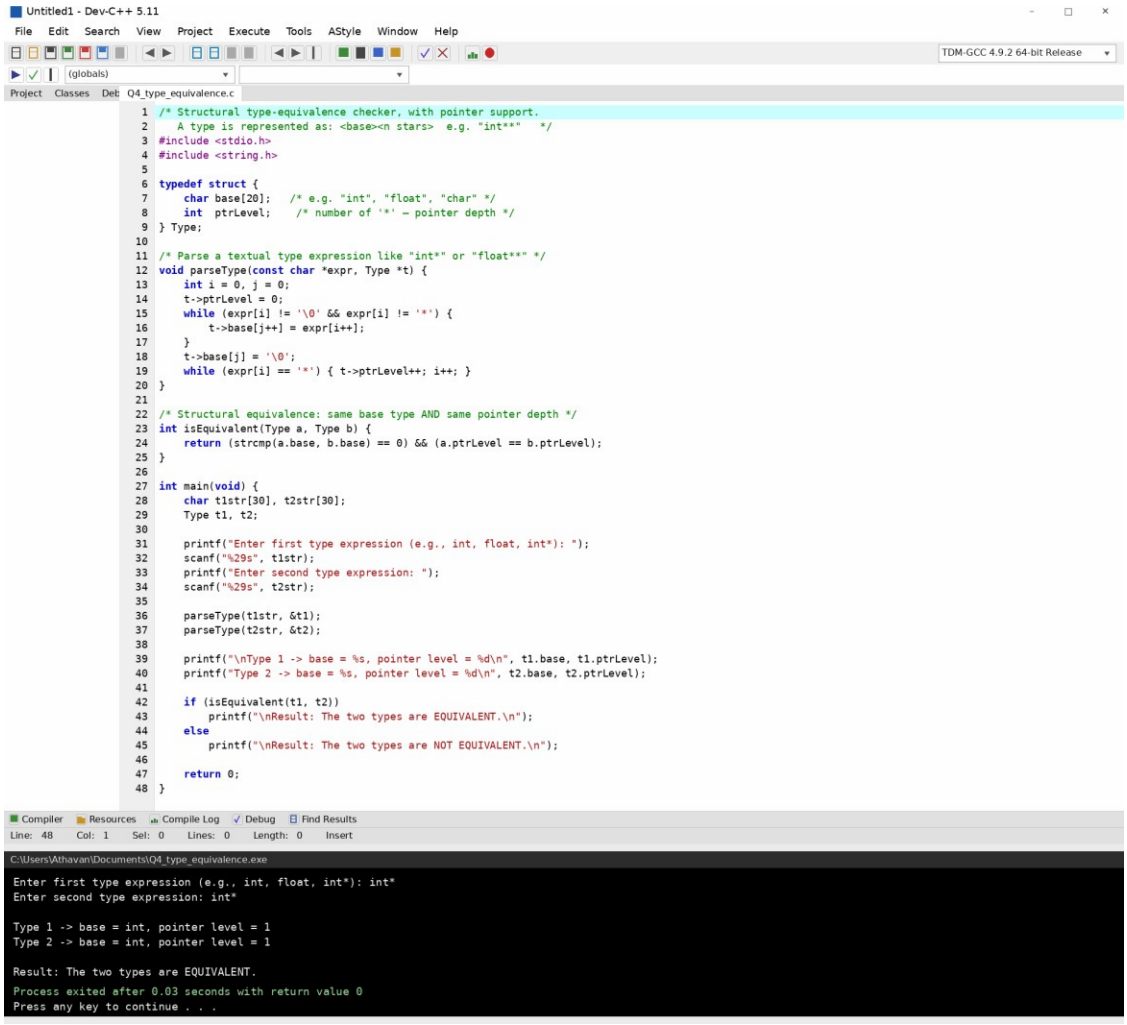
Program (Dev-C++) and Console Output:



Q4. Type Equivalence Checker (with Pointer Types)

- Accepts two type expressions (e.g., int, float, int*).
- Compares them structurally for type equivalence (base type + pointer depth).
- Displays whether the types are Equivalent or Not Equivalent, extended to handle pointer types.

Program (Dev-C++) and Console Output:



Q5. Semantic Analyzer — Type Checking of Arithmetic Expressions

- Accepts two operands and their types.
- Checks whether the operation (e.g., +, *) is valid between the given types.
- Reports a Valid expression or a Type error (e.g., int + char*), demonstrating semantic-error detection.

Program (Dev-C++) and Console Output: